Run MoE Models Bigger Than Your RAM

moe-cache makes stock llama.cpp 2–12× faster when a Mixture-of-Experts model doesn't fit in memory, and the output tokens stay identical. It's a plugin for an unmodified `llama-server`, with no fork and no patch. It keeps the experts you're actually using in RAM and streams the rest off your SSD, so a 102 GB GLM-5.3-Flash runs on a 24 GB laptop at 5.5× stock speed. [1]

12×	4×	0	13
Peak speedup gpt-oss-120b, 0.37 → 4.47 tok/s	Model vs. memory 102 GB model under a 24 GB limit	Changed tokens Same output as stock on every tested model	Releases in 8 days v0.1 on Oct 8 → v0.2.3.5 on Oct 15

I. The Speed Cliff, Measured

1. Decode speed, stock vs. moe-cache

Tokens per second on a laptop (i7-13620H, RTX 4060 8 GB, NVMe), with memory capped by a cgroup. Every model here is bigger than the RAM it's allowed to use.

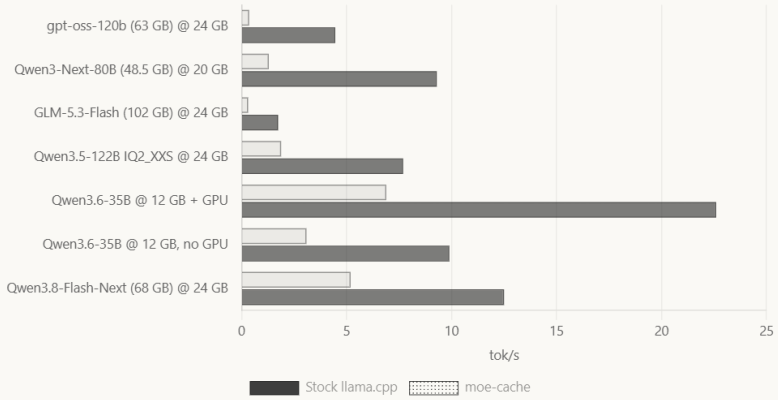


Fig. 1 — Exact mode (identical tokens) only. `--fast` and `--hybrid` go higher. [2]

2. Every result in one table

MODEL, MEMORY LIMIT	STOCK	MOE-CACHE	GAIN
gpt-oss-120b (63 GB), 24 GB	0.37	4.47	12.1×
Qwen3-Next-80B (48.5 GB), 20 GB	1.3	9.3	7.2×
GLM-5.3-Flash (102 GB), 24 GB	0.32	1.75 (2.1 fast)	5.5×
Qwen3.5-122B IQ2_XXS / IQ3_XXS / IQ4_XS, 24 GB	1.89 / 1.10 / 0.91	7.7 / 5.7 / 3.7	4–5×
Qwen3.6-35B (21.7 GB), 12 GB + GPU	6.9	22.6	3.3×
Qwen3.6-35B, 12 GB, CPU only	3.1	9.9	3.2×
Qwen3.8-Flash-Next IQ2_XS (68 GB), 24 GB	5.2	12.5 (17.6 hybrid)	2.4×
Prefill: Flash-Next, 2–5k prompts	65.9	90–103	1.4–1.6×

- Before every capped stock run, the model file is dropped from the page cache. Otherwise stock reads from uncounted cache and looks up to 4.6× faster than it really is.
- The gpt-oss and Qwen3-Next rows come from v0.1. The others were re-measured on v0.2.3.x.

II. Why It's Faster

1. The problem with mmap

MoE models use only a few experts per token, but llama.cpp memory-maps all of them. Once the file is bigger than RAM, the OS page cache thrashes and throughput collapses. moe-cache takes over the CPU-side expert matrix multiplications and decides for itself which experts have real memory behind them.

2. What the plugin does per token

- It reads the router's expert picks, then loads any missing experts from the SSD with large, parallel direct reads.
- The expert executor computes the experts already in RAM while the rest are still loading, and it can use Intel efficiency cores too.
- It sizes the cache from your real memory limit and evicts least-recently-used experts, so it doesn't get OOM-killed.
- `moe-cache-learn` records which experts you use and warm-starts from that profile next time.
- It uses llama.cpp's own dot-product kernels. The one exception is a new SIMD Q2_0 kernel that's bit-identical and about 3× faster on x86.

3. Identical, and checked

VERIFIED

The default mode gives the same tokens as stock llama.cpp, and CI checks that on every push. The identity workflow runs Granite 3.1 with forced eviction (about 39k evictions) and with the executor byte-checked (6,708 ops, 0 mismatches). It passes 4/4 on both supported ggml APIs. The sanitizer workflow runs the whole plugin under ASan and UBSan. A race that could break exactness was found and fixed in v0.2.3.3, with a new test that fails the old build. That fix is recorded openly in the changelog. [3]

4. Modes beyond the default

Exact is the default. Every other mode is opt-in and labelled with what it costs.

MODE	WHAT IT DOES	MEASURED EFFECT	EXACT?
Default	RAM cache + SSD streaming + executor	Fig. 1 numbers	Yes
`--gpu-cache` `auto`	Works with llama.cpp's own GPU expert cache	Flash-Next 12.5 → 13.1 tok/s	Yes, vs stock + cache
`--fast`	Skips a low-ranked expert that isn't in RAM	+10–12%; GLM 1.75 → 2.1, Qwen3.5-122B 7.7 → 9.0	No: ~1 in 40–50 top tokens differ
`--hybrid`	The most-used experts live in a GPU bank, computed in parallel with the CPU	Flash-Next up to 17.6 tok/s	No: ~1 in 60 top tokens differ
`--project NAME`	Saves a usage profile and prompt state for a project	2.7k-token context restored in 0.25 s instead of 56 s	Yes

- `--hybrid` is experimental and needs a usage profile first. It's slower than `--gpu-cache` on Q4_K models such as Qwen3.6.
- Five experimental switches that lost in measurement were removed in v0.2.3.5, not left in as dead options.

III. Don't Trust These Numbers: Run Them

1. moe-cache-benchmark

SEPARATE REPO

One laptop's numbers don't prove much, so the benchmark tool lives in its own repo for anyone to run. It compares stock llama.cpp and moe-cache on your machine, with your model and your prompts, under a memory limit you choose, and checks that both produce the same tokens. [5]

- `bin/moe-cache-bench` opens a local web page: pick a .gguf, set memory limits (e.g. 12 and 24 GB), and press Start.
- It runs in both orders to cancel out drift, and it flags runs that started hot or under load.
- It reports steady decode speed, time to first token, SSD reads per token, cache hit rate, and PASS/FAIL for identical tokens.
- You can add your own prompts, with `EXPECT:` words the answer must contain.
- `bench/run_matrix.py --ollama --dir ~/models --run` benchmarks every model on your disk and writes a table you can publish.
- Shareable reports leave out prompts, outputs and local paths. It needs Linux, Python 3 (standard library only) and systemd-run.

IV. When Not to Use It

1. Honest boundaries

- **The model fits in RAM:** stock wins, 42.9 vs 36.6 tok/s on Qwen3.6-35B at 24 GB. The launcher detects this case and starts stock llama-server instead.
- **You want peak speed on Flash-Next or Qwen3.6 with 12 GB+ VRAM:** Strata, a single-model engine, reports 94 tok/s on an RTX 5070.
- **Your model is bigger than RAM, uses another MoE architecture, or you want to stay on stock llama.cpp:** this is the case moe-cache is built for.

V. Where It Stands Today

1. Platform support

PLATFORM	STATUS
Linux + Intel + NVIDIA (CUDA)	Tested
Linux + Vulkan (NVIDIA, Intel iGPU)	Tested on llama.cpp 7fe450e19
Linux, CPU only	Tested; identical, within ~4% of stock when the model fits
AMD GPU / CPU	Expected to work, not yet tested
Windows	In progress (see below)
macOS	No

2. Windows: now in CI

IN PROGRESS

Windows CI went live on Oct 15 and took seven runs to go green. The Windows layer was already cross-compiled with mingw-w64 and passed its tests under Wine. Moving the identity test onto a real GitHub Windows runner surfaced real bugs. [4]

- Runs #1 and #2 failed. The fix made plugin stderr unbuffered, so crashes leave log lines behind.
- #4 and #5 failed because the plugin repacked weights despite `--no-repack`, and only 1 of 4 outputs was identical.
- #7 passed, and the docs now report forced eviction 4/4 identical with the executor byte-checked on the runner.
- What's still missing: no one has run it on a physical Windows machine yet, and there's no Windows launcher, so the README still lists Windows as unsupported.

3. Road to v1.0

- Real Windows hardware, and a first AMD run (a Ryzen mini PC

is waiting).
- CI that runs real models for identity checks and a speed regression guard. Today CI runs a small Granite model only.
- Tracking newer llama.cpp releases. Backend API 3 is first-class, and API 2 gets bug fixes only.

VI. Built in the Open

1. How it was made DISCLOSURE

moe-cache was written with heavy help from Claude, and the commits are co-authored with it. The ideas, testing and measurements are the author's own. Every number in this Report was run on the author's laptop, and the methods are documented in RESULTS.md. Because AI-assisted code deserves extra scrutiny, the project leans hard on verification: token-identity checks against stock in CI, ASan, UBSan and TSan runs, and mutation builds that the tests must catch.

2. If anything looks off

Nothing here is hidden, and corrections are welcome. If a number doesn't add up, a claim seems overstated, the code does something odd, or you can't reproduce a result, open a GitHub issue or message the author directly. Everything is MIT-licensed and public.

VII. Try It

1. Three commands

- Install from a prebuilt release (attested with `gh attestation verify`), or run `./install.sh` on a fresh Ubuntu or Debian box.
- Check your setup: `bin/moe-cache-check model.gguf --llama-server PATH` prints OK, WARN or FAIL for each line.
- Run it: `bin/moe-cache-server model.gguf --ram 12 --open` serves an OpenAI-compatible API on localhost:8080.
- Have AMD or Windows hardware? A run of `tests/identity.sh` or the benchmark tool on your machine is the most useful thing you can contribute.

Made with Dia